

Retrieval-Augmented Generation for Large Language Models in the Vietnamese Legal Domain

Thanh Tung

August 2026

Contents

1 ĐẶT VẤN ĐỀ	2
2 GIỚI THIỆU	3
2.1 Tổng quan về RAG	3
2.2 Naive RAG	5
3 LÝ DO THỰC HIỆN ĐỀ TÀI	7
4 DATASET	7
5 RAG FRAMEWORK	8
5.1 Preprocessing	8
5.2 Indexing	9
5.2.1 Chunking	9
5.2.2 Embedding	10
5.3 Retrieval	11
5.4 Generation	12
5.4.1 Prompt Design	13
6 EXPERIMENT AND RESULT	13
6.1 Retrieval Evaluation	13
6.2 Generation Evaluation	15
7 CONCLUSION	16

1 ĐẶT VẤN ĐỀ

Large Language Models (LLMs) trong những năm gần đây đã cho thấy được sức mạnh vượt trội trong các tác vụ có liên quan tới việc xử lý và tạo ra ngôn ngữ giống người thật (human-like language), tuy nhiên bên cạnh đó vẫn gặp phải các hiện tượng, vấn đề như hallucination, outdated knowledge, non-transparent, untraceable reasoning process. Retrieval-Augmented Generation (RAG) là một giải pháp có thể giúp khắc phục hoặc hạn chế các vấn đề mà các LLMs phải đối mặt như đã đề cập từ trước bằng cách sử dụng các database từ bên ngoài như là một kho tri thức củng cố cho các LLMs. Việc sử dụng kỹ thuật này có thể cải thiện rõ rệt độ chính xác cũng như độ tin cậy của các thông tin được tạo ra từ LLMs, RAG đặc biệt hiệu quả đối với các tác vụ đòi hỏi kiến thức chuyên môn (domain specific information) cũng như cần phải liên tục cập nhật kiến thức.

2 GIỚI THIỆU

RAG đã phát triển với tốc độ nhanh trong những năm gần đây. Việc cải thiện RAG không còn chỉ giới hạn ở giai đoạn inference mà còn bắt đầu kết hợp với các kỹ thuật fine-tuning LLMs.

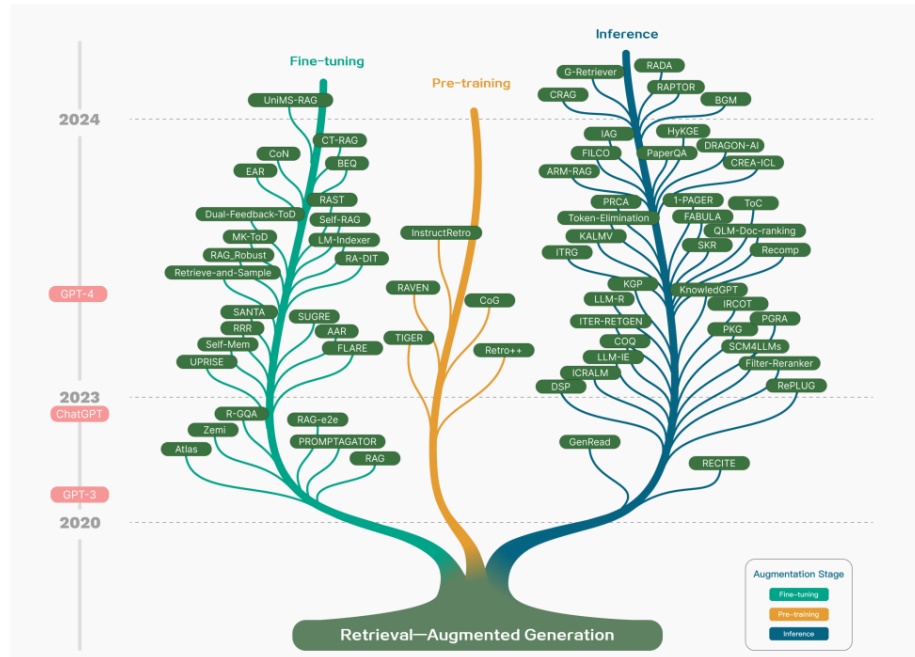


Figure 1: Technology tree của RAG research. Các giai đoạn sử dụng RAG chủ yếu là pre-training, fine-tuning và inference. Khi LLMs xuất hiện, việc nghiên cứu RAG ban đầu tập trung vào việc tận dụng khả năng học tập mạnh mẽ theo ngữ cảnh(context) của LLMs, chủ yếu vào quá trình suy luận. Về sau các nghiên cứu đã tiến xa hơn, dần dần đi vào việc fine-tuning các LLMs

2.1 Tổng quan về RAG

Một ứng dụng điển hình của RAG được mô tả trong hình 2. Một người dùng đặt câu hỏi cho ChatGPT về một tin tức gần đây được nhiều người bàn tán. Vì GPT dựa vào quá trình pre-train nên sẽ thiếu khả năng cung cấp thông tin về những dữ kiện gần đây. RAG khắc phục khoảng trống thông tin này bằng cách tìm kiếm và kết hợp thông tin từ các cơ sở dữ liệu bên ngoài. Trong trường hợp này nó thu thập các bài báo có liên quan tới truy vấn của người dùng, bằng cách này có thể kết hợp với câu hỏi gốc để tạo thành một chỉ dẫn đủ chi tiết để hướng dẫn cũng như cung cấp đầy đủ ngữ cảnh giúp LLMs đưa ra câu trả lời có cơ sở rõ ràng.

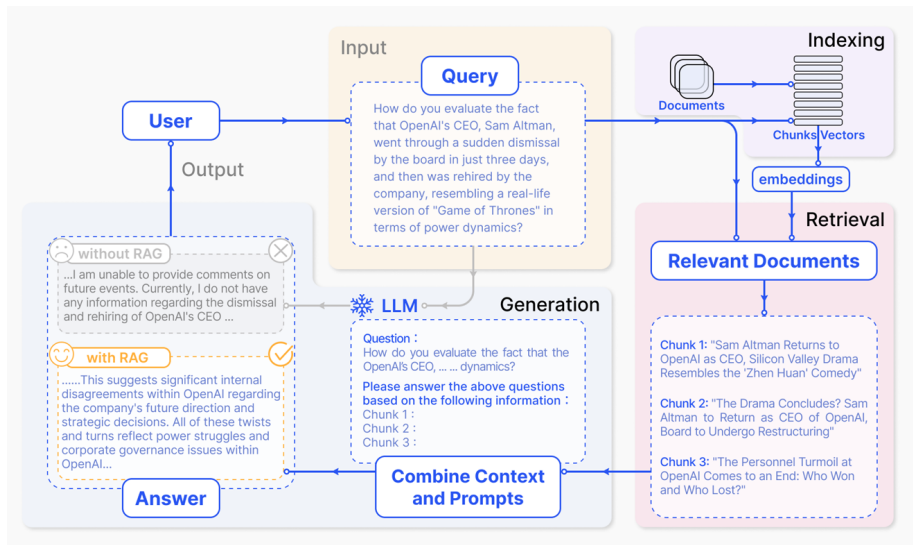


Figure 2: Một pipeline điển hình của RAG, gồm 3 bước chính. 1) Indexing. Tài liệu sẽ được chia thành các chunks, mã hóa thành các vectors, và lưu trữ trong database(vector database). 2) Retrieval. Retrival top K chunks liên quan nhất tới câu hỏi dựa trên độ tương đồng về mặt ngữ nghĩa. 3) Generation. Đưa câu hỏi gốc và các phần lấy được từ retrieval vào LLMs để tạo ra câu trả lời cuối cùng.

Việc nghiên cứu RAG vẫn đang liên tục phát triển và có thể phân loại nó thành 3 giai đoạn: Naive RAG, Advance RAG, và Modular RAG.

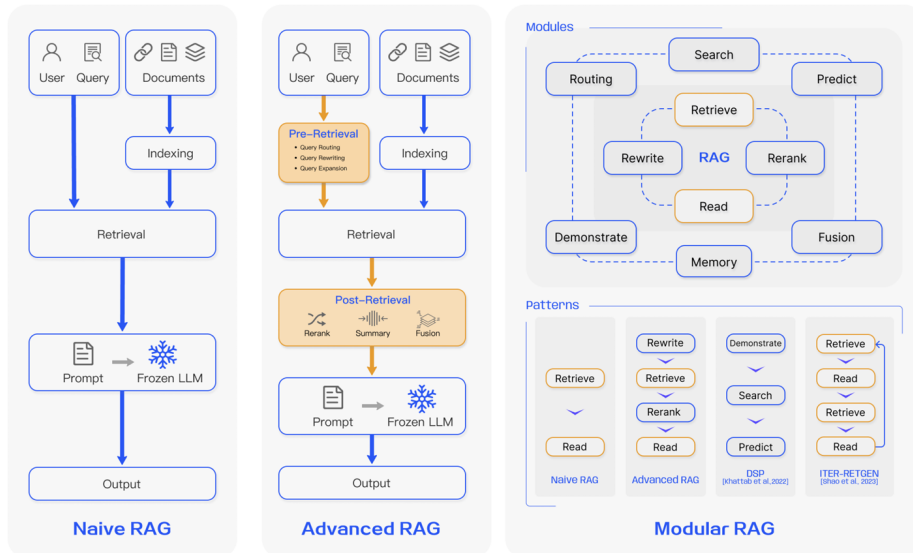


Figure 3: Các cấp độ phát triển RAG

Trong phạm vi của project này nhóm sẽ thực hiện Naive RAG.

2.2 Naive RAG

Naive RAG chính là phương pháp sớm nhất, trở nên nổi bật sau khi GPT trở nên phổ biến. Naive RAG theo quy trình truyền thống gồm 3 bước indexing, retrieval, và generation, còn được mô tả như là một "Retrieve-Read" framework.

Indexing. Bắt đầu bằng việc làm sạch và trích xuất dữ liệu thô từ nhiều định dạng khác nhau như PDF, HTML, Word, và Markdown và chuyển thành dạng text. Để phù hợp với giới hạn context của language models, văn bản được chia thành các phần nhỏ gọi là chunks. Các chunk này sau đó được encode thành các vector bằng cách sử dụng các embedding model và lưu dưới dạng vector database. Đây là một bước quan trọng để cho việc tìm kiếm và truy xuất dựa trên sự tương đồng về sau.

Retrieval. Khi nhận được truy vấn từ user, hệ thống RAG sử dụng cùng 1 embedding model để encode query thành vector. Sau đó, tiến hành tính điểm tương đồng giữa query vector và các vector của các chunk đã được index trong corpus. RAG sẽ ưu tiên và truy xuất top K chunks có độ giống với truy vấn cao nhất. Những đoạn này sau đó được sử dụng để làm ngữ cảnh mở rộng cho prompt.

Generation. Query và các chunks được chọn kết hợp thành một prompt thống nhất để LLMs sinh ra câu trả lời. Các tiếp cận của các LLMs có thể khác nhau tùy theo tiêu chí cụ thể của từng task, cho phép nó sử dụng kiến thức đã được pre-train hoặc giới hạn câu trả lời chỉ trong các thông tin có trong các tài liệu được cung cấp. Trong các trường hợp đối thoại liên tục, lịch sử trò chuyện hiện có có thể được tích hợp vào prompt, giúp mô hình tham gia hiệu quả vào các cuộc đối thoại nhiều lượt.

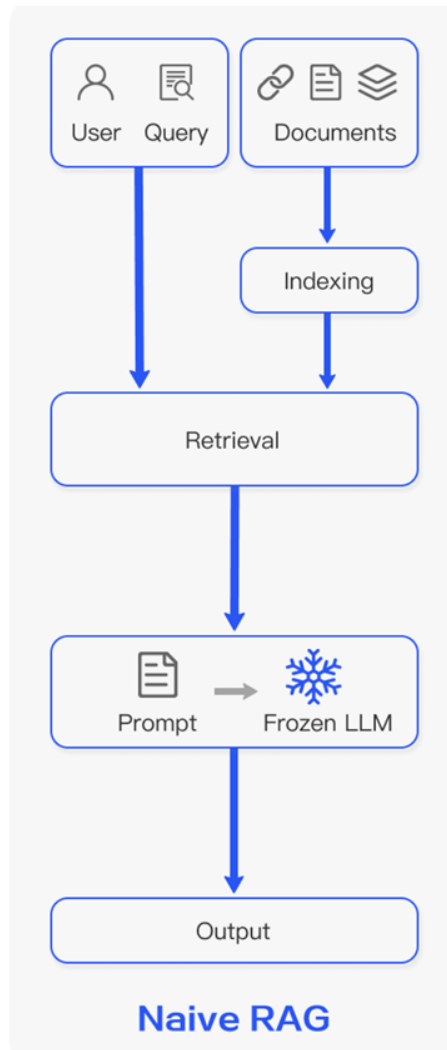


Figure 4: Sơ đồ của Naive RAG

Tuy nhiên, Naive RAG vẫn gặp những hạn chế đáng kể:

Retrieval Challenges. Giai đoạn retrieval thường gặp khó khăn với precision và recall, dẫn đến việc chọn nhầm các chunks không phù hợp hoặc không liên quan, và bỏ lỡ thông tin quan trọng.

Generation Difficulties. Khi tạo phản hồi, model có thể đối mặt với vấn đề 'hallucination', tức là tạo nội dung không được hỗ trợ bởi context truy vấn. Giai đoạn này cũng có thể gặp phải tình trạng về câu trả lời có thể không liên quan, độc hại, hoặc thiên kiến trong các phản hồi, làm giảm chất lượng và độ tin cậy của các câu trả lời.

Augmentation Hurdles. Việc tích hợp thông tin đã được truy xuất với các nhiệm vụ khác nhau có thể khá thách thức, đôi khi dẫn đến những kết quả rời rạc hoặc không mạch lạc. Quá trình này cũng có thể gặp phải sự dư thừa khi những thông tin giống nhau được lấy từ nhiều nguồn, dẫn đến các phản hồi lặp đi lặp lại. Việc xác định tầm quan trọng và sự liên quan của các đoạn văn khác

nhau cũng như đảm bảo tính nhất quán về văn phong và giọng điệu càng làm tăng sự phức tạp hơn. Khi đối mặt với những vấn đề phức tạp, việc chỉ dựa vào một lần truy suất dựa trên truy vấn gốc có thể không đủ để có được thông tin bối cảnh đầy đủ.

3 LÝ DO THỰC HIỆN ĐỀ TÀI

Như đã đề cập ở trên LLMs thường gặp các vấn đề với những domain chuyên biệt, đòi hỏi độ chính xác cao và có tính cập nhật liên tục, tiêu biểu là lĩnh vực pháp lý. Trong pháp luật nếu không liên tục cập nhật thông tin mà chỉ dựa vào quá trình pre-train có thể dẫn tới việc một câu trả lời hợp lý về mặt ngữ nghĩa nhưng thời gian quy định đã hết hiệu lực hoặc điều luật đã được sửa đổi. RAG không chỉ giúp khắc phục các vấn đề của LLMs mà còn tạo điều kiện để câu trả lời được tham chiếu vào nguồn tài liệu cụ thể và chính xác đặc biệt trong hoàn cảnh hệ thống pháp lý liên tục biến động với các thông tư, nghị định được sửa đổi mỗi ngày. RAG cho phép hệ thống cập nhật thông tin tức thì qua việc làm mới kho dữ liệu mà không cần trải qua quá trình tái huấn luyện (retraining) tốn kém.

4 DATASET

Nhóm sử dụng dataset Vietnamese Legal Q&A[2] để áp dụng cho RAG và Fine-tuning LLMs. Đây là bộ dữ liệu gồm 4837 cặp câu hỏi - câu trả lời tình huống pháp lý, được sinh tự động bởi AI dựa trên 5 bộ luật cơ bản của Việt Nam:

1. Luật Doanh nghiệp 2020
2. Bộ luật Dân sự 2015
3. Luật Hôn nhân và Gia đình 2014
4. Bộ luật Lao động 2019
5. Bộ luật Hình sự 2017

Table 1: Thống kê phân bố câu hỏi theo 5 bộ luật

Tên bộ luật	Số lượng câu hỏi	Tỷ lệ (%)
Luật Doanh nghiệp 2020	654	13.52
Bộ luật Dân sự 2015	2062	42.63
Luật Hôn nhân và Gia đình 2014	382	7.90
Bộ luật Lao động 2019	580	11.99
Bộ luật Hình sự (Văn bản hợp nhất 2017)	1159	23.96

Cấu trúc dữ liệu bao gồm:

- **question:** Câu hỏi tình huống
- **law_content:** Nội dung điều luật gốc

- `law_id`: Số hiệu điều luật
- `law_name`: Tên bộ luật

5 RAG FRAMEWORK

Như trong hình 4, framework của RAG được chia thành 3 modules chính:

- *Indexing*: Phân tách chunks, encode thành vector và lưu dưới dạng vector database.
- *Retrieval*: Truy xuất top K chunks có độ tương đồng cao nhất với query.
- *Generate*: Tích hợp chunks truy suất vào câu hỏi để cung cấp ngữ cảnh cho LLMs.

5.1 Preprocessing

Giai đoạn có nhiệm vụ xây dựng kho dữ liệu(corpus) các điều luật duy nhất, có định danh(`doc_id`) và dễ đọc, làm cơ sở cho retrieval.

Trước khi xây dựng corpus nhóm đã nhận thấy một số vấn đề như sau:

1. Mối quan hệ giữa câu hỏi và điều luật: Một câu hỏi có thể liên kết với nhiều điều luật khác nhau
2. Có sự trùng lặp trong `law_content`: Một điều luật có thể trả lời cho nhiều câu hỏi

Table 2: Thống kê mô tả các trường dữ liệu

Trường dữ liệu	Số lượng	Giá trị duy nhất
ID	4,837	2,839
Câu hỏi	4,837	4,837
Nội dung điều luật	4,837	1,675
ID điều luật	4,837	689
Tên bộ luật	4,837	5

Khi tiến hành xây dựng corpus, thành viên đã tiến hành các bước sau:

1. Loại bỏ những điều luật trùng lặp
2. Với mỗi điều luật cần có một khóa định danh chính(`doc_id`) được format như sau: `ten-dieu-luat_dieu-xxx`
3. Tạo corpus file với các trường dữ liệu: `doc_id`, `law_name`, `law_id`, `law_content`

Sample mẫu của corpus:

STT	Doc ID	Law Name	Law ID	Law Content
0	luat-doanh-nghiep-2020_dieu-1	Luật Doanh nghiệp 2020	Điều 1	Điều 1. Phạm vi điều chỉnh. Luật này quy định về việc thành lập, tổ chức quản lý, tổ chức lại, giải thể và hoạt động có liên quan của doanh nghiệp, bao gồm công ty trách nhiệm hữu hạn, công ty cổ phần, công ty hợp danh và doanh nghiệp tư nhân; quy định về nhóm công ty.
1	luat-doanh-nghiep-2020_dieu-2	Luật Doanh nghiệp 2020	Điều 2	Điều 2. Đối tượng áp dụng. 1. Doanh nghiệp. 2. Cơ quan, tổ chức, cá nhân có liên quan đến việc thành lập, tổ chức quản lý, tổ chức lại, giải thể và hoạt động có liên quan của doanh nghiệp.

Table 3: Dữ liệu văn bản pháp luật đã được chuẩn hóa đưa vào corpus

5.2 Indexing

5.2.1 Chunking

Vì mỗi điều luật là một văn bản hoàn chỉnh và có ý nghĩa độc lập, độ dài được thống kê trong bảng dưới đây cho thấy độ dài trung bình vừa phải, phù hợp để giữ nguyên mỗi 1 Điều luật = 1 Vector mà không cần chunking thêm.

Table 4: Thống kê độ dài theo từ của điều luật

Statistic	Word Length
Count	1,675
Mean	208.09
Std	526.06
Min	21.00
25%	86.00
50%	146.00
75%	247.50
Max	20,460.00

5.2.2 Embedding

Trước khi đưa vào mô hình embedding thành viên đã ghép thêm tiêu đề bộ luật (`law_name`) vào trước nội dung của bộ luật (`law_content`), điều này giúp Embedding model có thể hiểu được ngữ cảnh bao quát của văn bản (đoạn này thuộc luật nào), giảm nguy cơ nhầm lẫn giữa các điều luật có nội dung tương tự nhau ở các bộ luật khác nhau.

Format của document embedding:

```
enriched_text = (  
    f"Văn bản: {item['law_name']}\n"  
    f"Nội dung: {item['law_content']}"  
)
```

Thành viên lựa chọn BAAI/bge-m3[3] làm kiến trúc embedding bởi các ưu điểm sau:

- Khả năng hỗ trợ tiếng việt
- Giới hạn độ dài ngữ cảnh vượt trội lên đến 8.192 tokens, cho phép mã hóa toàn bộ văn bản của từng điều luật mà không làm đứt gãy ngữ cảnh
- Đầu ra vector được chuẩn hóa (L_2 normalization) giúp tối ưu hóa hiệu năng tính toán độ tương đồng Cosine trên cơ sở dữ liệu vector.

ChromaDB được lựa chọn làm Vector Database bởi các ưu điểm sau:

- Chỉ mục ANN (Approximate Nearest Neighbor): ChromaDB tích hợp sẵn thuật toán đồ thị HNSW (Hierarchical Navigable Small World), cho phép tìm kiếm các vector có độ tương đồng cao nhất (Cosine Similarity / Inner Product) với độ phức tạp thời gian chỉ khoảng $\mathcal{O}(\log N)$, vượt trội hoàn toàn so với việc quét tuyến tính ($\mathcal{O}(N)$).
- Tính toán khoảng cách tối ưu: Tương thích trực tiếp với các vector đã được chuẩn hóa L_2 từ các mô hình embedding (như BGE-M3), đảm bảo kết quả truy xuất chính xác theo không gian ngữ nghĩa.
- Metadata Filtering song song: ChromaDB cho phép đính kèm metadata có cấu trúc (như `law_name`, `law_id`, `doc_id`) trực tiếp vào từng vector.

Bên cạnh việc sử dụng vector embedded nhóm còn sử dụng thêm thuật toán BM25 (Best Matching 25) để tiến hành index theo từ khóa (Sparse Indexing). Phương pháp này cho phép hệ thống truy hồi các tài liệu có mức độ tương đồng cao với truy vấn dựa trên sự xuất hiện và trọng số của các thuật ngữ. Đối với dữ liệu pháp luật, sparse indexing đặc biệt hữu ích trong việc truy hồi các thuật ngữ chuyên ngành, số hiệu điều luật, khoản và các cụm từ pháp lý có tính định danh cao.

Tóm gọn lại, trong project này nhóm đang xây dựng một hệ thống Hybrid Retrieval dựa trên Dense Indexing và Sparse Indexing:

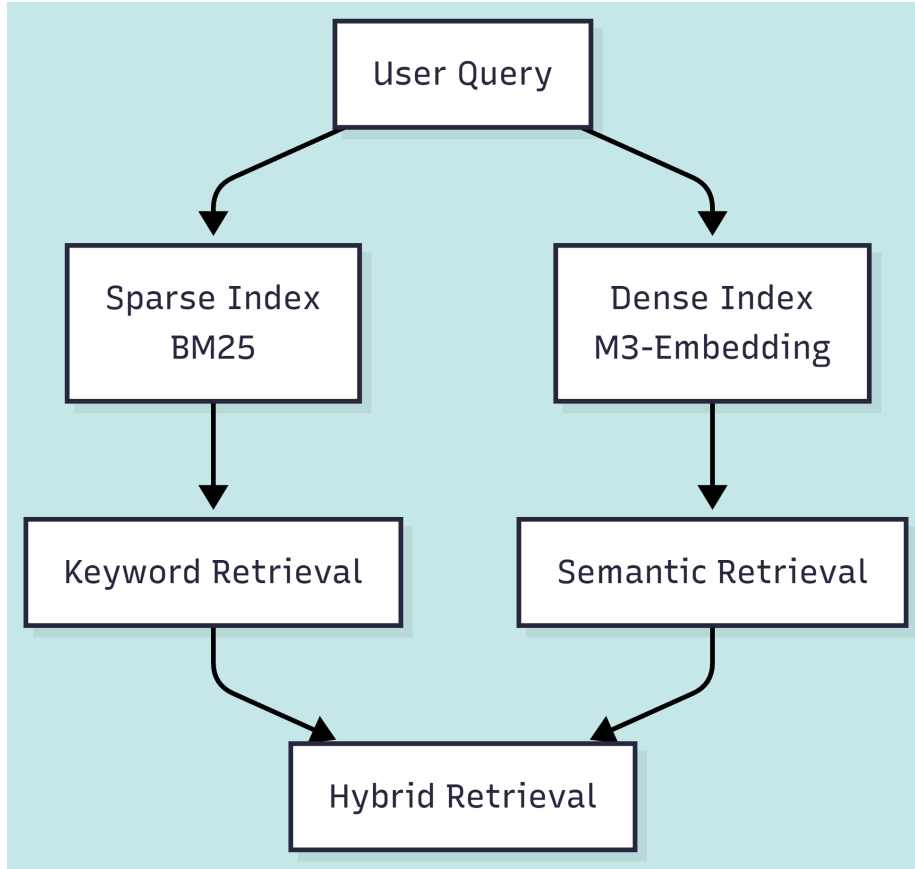


Figure 5: Sơ đồ Hybrid Retrieval

5.3 Retrieval

Sau khi indexing corpus, tiếp theo là giai đoạn Retrieval.

Retrieval được chia thành 3 phần chính:

1. Hybrid Retrieval:
 - *Dense Retrieval*: Câu hỏi được mã hóa thành vector thông qua mô hình embedding (BAAI/bge-m3) và truy vấn trên ChromaDB để lấy ra Top- K tài liệu tương đồng về ngữ nghĩa.
 - *Sparse Retrieval*: Câu hỏi được tách từ và chấm điểm qua mô hình BM25 để lấy ra Top- K tài liệu khớp chính xác các từ khóa/thuật ngữ pháp lý.
2. Rank Fusion Strategy: Sử dụng thuật toán Reciprocal Rank Fusion (RRF) để kết hợp điểm số và thứ hạng của 2 bộ kết quả mà không cần chuẩn hóa thang đo điểm số khác nhau:

$$RRF_Score(d \in D) = \sum_{m \in M} \frac{1}{k + r_m(d)}$$

Trong đó:

- M : Tập các phương pháp truy xuất (BM25, Dense).
 - $r_m(d)$: Thứ hạng của tài liệu d trong danh sách kết quả của phương pháp m .
 - k : Hằng số làm mượt (thường chọn $k = 60$).
3. Reranking: Cross-Encoder (BAAI/bge-reranker-v2-m3) được sử dụng để chấm điểm lại Top- N tài liệu sau bước RRF nhằm nâng cao độ chính xác trước khi gửi vào ngữ cảnh cho LLM.

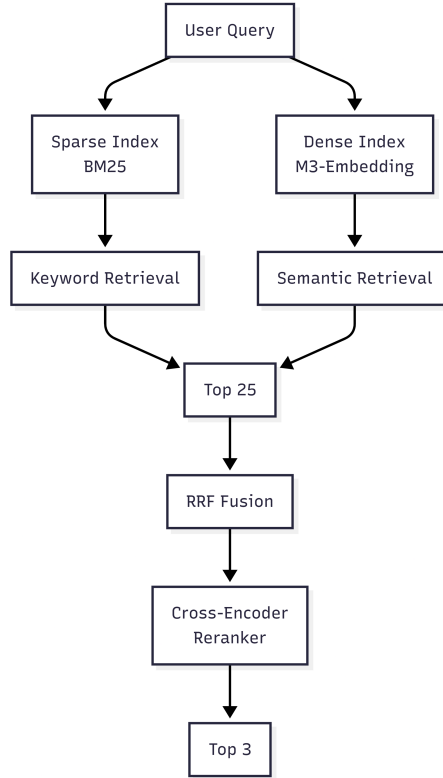


Figure 6: Retrieval Pipeline

5.4 Generation

Nhóm sử dụng model **Qwen/Qwen2.5-3B-Instruct** bởi các ưu điểm sau:

- Tạo văn bản dài (trên 8k token), hiểu dữ liệu có cấu trúc (ví dụ: bảng biểu) và tạo đầu ra có cấu trúc, đặc biệt là định dạng JSON.
- Hỗ trợ ngữ cảnh dài lên tới 128.000 token và khả năng tạo văn bản dài tới 8.000 token.
- Hỗ trợ đa ngôn ngữ với hơn 29 ngôn ngữ, bao gồm tiếng Việt.

5.4.1 Prompt Design

Prompt được thiết kế với ba thành phần chính. Thứ nhất, hệ thống cung cấp vai trò cho LLM với tư cách là một chuyên gia tư vấn pháp luật. Thứ hai, các văn bản được truy hồi được đưa vào prompt dưới dạng các khối ngữ cảnh riêng biệt và được đánh số để mô hình có thể phân biệt giữa các tài liệu. Thứ ba, hệ thống yêu cầu LLM trả lời theo cấu trúc gồm ba phần: căn cứ pháp lý, nội dung quy định và phân tích & hướng dẫn.

Prompt được nhóm sử dụng có cấu trúc như sau:

Bạn là một chuyên gia tư vấn pháp luật.

Dựa trên các quy định của pháp luật Việt Nam được cung cấp dưới đây, hãy giải đáp câu hỏi của người dùng và bắt buộc trình bày theo đúng định dạng 3 phần:

****Căn cứ pháp lý:****

[Tên điều luật] - [Tên văn bản luật]

****Nội dung quy định:****

[Trích dẫn chính xác nội dung điều luật áp dụng]

****Phân tích & Hướng dẫn:****

[Phân tích và áp dụng quy định vào tình huống của người dùng]

CĂN CỨ PHÁP LUẬT THAM KHẢO:

--- [Tài liệu 1] ---

{document_1}

--- [Tài liệu 2] ---

{document_2}

...

--- [Tài liệu K] ---

{document_K}

CÂU HỎI:

{query}

TRẢ LỜI:

6 EXPERIMENT AND RESULT

6.1 Retrieval Evaluation

Metrics:

- Hit Rate @ K: Với một query q , giả sử retriever trả về Top-K documents:

$$D_q = [d_1, d_2, \dots, d_K]$$

Nếu ít nhất một document relevant xuất hiện trong Top-K, ta coi query đó là một hit. Sau đó lấy trung bình trên toàn bộ N queries:

$$\text{HitRate@K} = \frac{1}{N} \sum_{i=1}^N \text{Hit}(q_i, K)$$

- MRR-Mean Reciprocal Rank: Chỉ số MRR đánh giá vị trí của relevant document trong top K:

$$RR(q) = \frac{1}{\text{rank}(q)}$$

MRR là trung bình:

$$\frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}$$

Table 5: Kết quả ablation study của các cấu hình retrieval

Config.	Hit@1	Hit@3	Hit@5	MRR@5
Dense only	56.61	73.55	78.31	0.653
Sparse only	32.64	49.38	59.09	0.424
Hybrid without Reranker	51.86	68.60	75.62	0.610
Hybrid with Reranker	67.36	84.71	86.78	0.755

Dựa vào bảng trên có thể đưa ra một số kết luận như sau:

- Dense only có kết quả tốt hơn so với sparse và hybrid: Dense only (Hit@1 56.61) vượt Hybrid without Reranker (Hit@1 51.86) ở mọi chỉ số mặc dù hybrid sử dụng cả dense và sparse. Giả thuyết cho kết quả này có thể nằm ở RRF khi nhóm chia đều trọng số RFF (dense_weight = sparse_weight = 0.5) khiến cho sparse với khả năng hit rate thấp ảnh hưởng ngang với dense.
- Sparse có kết quả thấp nhất. Nguyên nhân nhóm đưa ra là các câu query thường được diễn đạt bằng ngôn ngữ tự nhiên ví dụ như: "Vợ tôi đã phát hiện tôi có quan hệ tình cảm với một người khác và đã khiêu nại. Tôi đã bị xử phạt hành chính trước đó vì hành vi này. Liệu tôi có thể bị xử lý hình sự không?", dẫn đến việc khớp từ khóa thuần túy không hiệu quả bằng việc khớp ngữ nghĩa.
- Reranker đem lại kết quả cải thiện lớn khi được kết hợp cùng với hybrid search.

6.2 Generation Evaluation

Metrics:

- Citation Accuracy: Đo mức độ chính xác của các trích dẫn pháp lý trong câu trả lời.

$$\text{Citation Accuracy} = \frac{\text{Số câu trả lời có trích dẫn đúng ID/Tên điều luật chuẩn}}{\text{Tổng số mẫu kiểm thử } (N = 484)} \times 100\%$$

- Format Adherence: Đo xem câu trả lời có tuân thủ format đầu ra được quy định trong prompt hay không.

$$\text{Format Adherence} = \frac{\text{Số câu trả lời thỏa mãn 100\% quy tắc định dạng}}{\text{Tổng số mẫu kiểm thử } (N = 484)} \times 100\%$$

- ROUGE-L Average: ROUGE-L đo độ tương đồng giữa câu trả lời của LLM và câu trả lời tham chiếu dựa trên Longest Common Subsequence (LCS) – dãy con chung dài nhất.

$$R_{LCS} = \frac{\text{LCS}(\text{Reference}, \text{Generated})}{m}, \quad P_{LCS} = \frac{\text{LCS}(\text{Reference}, \text{Generated})}{n}$$

$$\text{ROUGE-L} = \frac{(1 + \beta^2)R_{LCS}P_{LCS}}{R_{LCS} + \beta^2P_{LCS}} \quad (\text{thường chọn } \beta = 1 \text{ để lấy } F_1\text{-Score})$$

(với m là số từ trong câu mẫu, n là số từ trong câu mô hình sinh).

Table 6: Ablation study Generation

Configuration	Citation Accuracy (%)	Format Adherence (%)	ROUGE-L (%)
Dense only	66.74	75.07	48.41
Sparse only	44.83	80.79	44.59
Hybrid without Reranker	61.36	77.13	48.20
Hybrid with Reranker	75.62	73.35	50.67

Dựa vào bảng trên có thể đưa ra một số kết luận như sau:

- Citation của Dense only vẫn cao hơn Sparse và Hybrid, phản ánh đúng với kết quả retrieval, retrieval thấp dẫn tới độ chính xác thấp.
- Format Adherence có xu hướng trái với Citation, trong khi citation của Sparse là thấp nhất thì Format Adherence của nhánh này lại có kết quả vượt trội so với các nhánh còn lại. Giả thuyết đưa ra cho vấn đề này có thể là do: khi context truy hồi được (từ sparse) ít liên quan nội dung, LLM có xu hướng "bám" chặt vào template/format được yêu cầu vì không có đủ nội dung thực chất để viết. Ngược lại khi context tốt (hybrid_with_rerank), model có nhiều nội dung pháp lý cụ thể để trích dẫn/diễn giải hơn, nên đôi khi lệch khỏi cấu trúc định dạng nghiêm ngặt để trình bày nội dung đầy đủ hơn. Đây cũng có thể được xem là 1 trade off để đánh đổi về mặt nội dung và độ chính xác format.
- ROUGE-L phản ánh đúng kết quả mong muốn: retrieval càng chính xác thì câu trả lời sinh ra càng gần với ground truth về mặt nội dung

7 CONCLUSION

Project đã xây dựng hoàn chỉnh một pipeline RAG cho bài toán hỏi-đáp pháp luật tiếng Việt, đi qua ba giai đoạn chính: indexing, retrieval và generation. Kết quả cuối cùng cho thấy, cấu hình tốt nhất: hybrid kết hợp dense, sparse và rerank bằng cross-encoder đạt Citation Accuracy 75,62%, ROUGE-L 50,67, cải thiện đáng kể so với các cấu hình đơn lẻ.

Bên cạnh đó cũng còn những hạn chế như sau:

- Corpus hiện chỉ bao phủ 5 văn bản luật, chưa đại diện cho toàn bộ hệ thống pháp luật, cũng như chưa có sự cập nhật các văn bản pháp luật hiện hành.
- Retrieval vẫn chưa thực sự tốt và cần được cải thiện thêm.

References

- [1] Y. Gao et al., “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2024. DOI: 10.48550/arXiv.2312.10997
- [2] Adamwhite265, *Vietnamese legal q&a*, Kaggle. [Online]. Available: <https://huggingface.co/datasets/adamwhite625/vietnam-legal-qa>
- [3] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation,” *arXiv preprint arXiv:2402.03216*, 2024. DOI: 10.48550/arXiv.2402.03216